

StepInputCard Documentation (Flutter)

This document explains the **StepInputCard** widget, its full source code, usage, and a complete step-based example. You can **edit this document directly**, **view it**, and **download it** from the canvas.

1. Overview

`StepInputCard` is a reusable Flutter UI component designed for **step-by-step input flows** such as: - Car selection - Multi-step forms - Onboarding flows

It includes: - Title & subtitle - Text input field - Progress indicator - Back, Clear, and Next/Done actions

2. Features

- Stateless & reusable widget
 - Step-based progress handling
 - Customizable actions (Back / Clear / Next)
 - Supports final-step confirmation (✓ icon)
 - Clean UI with Material Card
-

3. StepInputCard – Full Source Code

```
import 'package:nanduba/constants/textfontstyle.dart';
import 'package:nanduba/export.dart';

class StepInputCard extends StatelessWidget {
  final String title;
  final String subtitle;
  final String hintText;
  final bool isLastStep;

  final TextEditingController controller;
  final double progress;
  final VoidCallback? onBack;
  final VoidCallback onClear;
  final VoidCallback? onNext;

  const StepInputCard({
    super.key,
    required this.title,
```

```

    required this.subtitle,
    required this.hintText,
    required this.isLastStep,
    required this.controller,
    required this.progress,
    this.onBack,
    required this.onClear,
    this.onNext,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 1,
      color: AppColors.white,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(20),
      ),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            /// TITLE & SUBTITLE
            Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  title,
                  style: Textfontstyle.TextStyle12w500c212121poppins.copyWith(
                    color: AppColors.c343434,
                    fontWeight: FontWeight.bold,
                    fontSize: 12.sp,
                  ),
                ),
                Text(
                  subtitle,
                  style: Textfontstyle.TextStyle12w500c212121poppins.copyWith(
                    color: AppColors.c707070,
                    fontSize: 8.sp,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

```

```

const SizedBox(height: 12),

/// INPUT FIELD
CustomTextField(
  controller: controller,
  hintText: hintText,
  borderRadius: 16,
  readOnly: false,
),

const SizedBox(height: 14),

/// PROGRESS BAR
LinearProgressIndicator(
  borderRadius: BorderRadius.circular(8),
  value: progress,
  minHeight: 11,
  backgroundColor: AppColors.secondary,
  color: AppColors.primary,
),

2.height,

/// ACTION BUTTONS
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    /// BACK BUTTON
    if (onBack != null) ...[
      InkWell(
        onTap: onBack,
        child: Container(
          height: 40,
          width: 40,
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(8),
            color: AppColors.primary,
          ),
        child: const Icon(
          Icons.chevron_left,
          color: AppColors.white,
        ),
      ),
    ],
    2.width,
  ],

  /// CLEAR BUTTON

```

```

InkWell(
  onTap: onClear,
  child: Container(
    height: 40,
    width: 40,
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(8),
      color: AppColors.primary.withAlpha(20),
    ),
    child: const Icon(
      Icons.close,
      color: AppColors.primary,
    ),
  ),
),
2.width,

/// NEXT / DONE BUTTON
InkWell(
  onTap: onNext,
  child: Container(
    height: 40,
    width: 40,
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(8),
      color: AppColors.primary,
    ),
    child: Icon(
      isLastStep ? Icons.check : Icons.chevron_right,
      color: AppColors.white,
    ),
  ),
),
1,
),
1,
),
),
);
}
}

```

4. CarStep Model

```
class CarStep {  
    final String title;  
    final String subtitle;  
    final String hint;  
  
    CarStep({  
        required this.title,  
        required this.subtitle,  
        required this.hint,  
    });  
}
```

5. Step Configuration List

```
final List<CarStep> steps = [  
    CarStep(  
        title: "Select the car's manufacturer",  
        subtitle: "Make",  
        hint: "Vehicle Name",  
    ),  
    CarStep(  
        title: "Select the car's model",  
        subtitle: "Model",  
        hint: "Model Name",  
    ),  
    CarStep(  
        title: "Select the production year",  
        subtitle: "Year",  
        hint: "Production Year",  
    ),  
    CarStep(  
        title: "Select the variant",  
        subtitle: "Variant",  
        hint: "Variant",  
    ),  
    CarStep(  
        title: "Select the vehicle type",  
        subtitle: "Type",  
        hint: "Vehicle Type",  
    ),  
    CarStep(  

```

```

        title: "Select the chassis",
        subtitle: "Chassis",
        hint: "Chassis",
    ),
    CarStep(
        title: "Select the engine",
        subtitle: "Engine",
        hint: "Engine",
    ),
];

int currentStep = 0;

```

6. Usage Example (Stateful Widget)

```

StepInputCard(
  subtitle: steps[currentStep].subtitle,
  title: steps[currentStep].title,
  hintText: steps[currentStep].hint,
  controller: _nameController,
  progress: (currentStep + 1) / steps.length,
  isLastStep: currentStep == steps.length - 1,
  onBack: currentStep > 0
    ? () => setState(() {
        currentStep--;
        _nameController.clear();
      })
    : null,
  onClear: () {
    _nameController.clear();
  },
  onNext: () {
    if (currentStep < steps.length - 1) {
      setState(() {
        currentStep++;
        _nameController.clear();
      });
    } else {
      // Final submit action
    }
  },
);

```

7. How Progress Works

```
progress = (currentStep + 1) / steps.length;
```

- Automatically updates the progress bar
 - Final step shows ✓ instead of →
-

8. Customization Tips

- Replace `CustomTextField` with `Dropdown` or `BottomSheet`
 - Validate input before moving to next step
 - Store values per step using a `Map<int, String>`
 - Animate transitions with `AnimatedSwitcher`
-

9. Summary

`StepInputCard` is ideal for structured, guided user input flows. This documentation is **fully editable**, **viewable**, and **downloadable**.

You can now modify this document directly as needed.